

1 One-Way Permutations Imply UOWHF

From the end of lecture 10 we saw that the following hash function is 2-wise independent.

$$h_{a,b}(x) = (ax + b)_{1,n}$$

We extend this idea to create a UOWHF based on one way permutation f . First, we define the function $chop(x)$ to cut down a length n string to length $n - 1$ by deleting the first bit. The hash function h is defined as follows:

$$h_{a,b}(x) = chop(a(f(x)) + b)$$

Proof of Security: To demonstrate that this protocol is secure, we need to argue that an adversary that could break the Universal One-Way property of this function could break the one way permutation property of f . Consider an adversary A that could find another x' such that:

$$h_{a,b}(x) = chop(a(f(x')) + b)$$

Observe that there is only one other x' value that could map to the same $h_{a,b}(x)$ value. An adversary A' could have an advantage over the permutation f as follows.

1. Adversary A has the goal of finding inverse of value $y = f(x)$. Given oracle access to adversary A' , it receives value x' and solves for a, b :

$$ay + b = a(f(x')) + b$$

Since x was chosen at random, y must be random as well. Therefore, the values of a and b must be distributed randomly so when A returns hash function $h_{a,b}$ the function is indistinguishable from completely random h from H .

2. Pass the hash function $h_{a,b}$ to A' who finds x with non negligible probability. This is because there are only two solutions to $h_{a,b}(x)$ so the only collision it can find for h would be x' exactly. Meaning it can invert f with non-negligible probability, which is a contradiction.

1.1 Decreasing Hash Function

We claim that we can construct a hash function that maps $\{0, 1\}^{poly(n)} \rightarrow \{0, 1\}^n$ given a family of permutations $f_{poly(n)}, f_{poly(n)-1}, \dots, f_n$. The construction follows from the previous hash function, selecting $poly(n) - n$ random a, b values and at each step applying $x_{i-1} = chop(a_i f_i(x_i) + b_i)$. The proof

of security follows demonstrates that if this is not a secure hash function, then we can invert some f_k with non negligible probability.

Proof: Firstly observe that an adversary A to be able to invert h , it needs to find a collision at some level i for function f_i . Because of this, it must be able to find a collision for some function f_i with probability ϵ/n . We can invert this f_i by following the previous proof and replacing the output of level i with our new $a_i f(x') + b$. Now, if A outputs input to h that results in a collision, we can run the function and with probability at least $\epsilon/\text{poly}(n)$ it will invert f_k .

2 Encryption

2.1 Public Key Encryption

The modern definition of a Public Key Exchange (PKE) protocol is a triple of algorithms defined as follows:

- $\text{KeyGen}(1^k, R) \rightarrow pK, sK$
- $\text{Enc}(m, pK, R) \rightarrow C$
- $\text{Dec}(c', pK, sK) \rightarrow m'$

The informal definition is that nobody can get m given c without the secret key.

2.1.1 Goldwasser-Micali

Goldwasser and Micali published a paper describing properties that a cryptographic protocol should have. Among these, there is the notion of knowing nothing about the plaintext given the ciphertext. It should be the case that even an encryption of a single bit should be indistinguishable whether its a 0 or 1, which immediately implies that there must be some notion of randomness used.

2.2 Diffie-Hellman Key Agreement

2.2.1 Discrete Log

The Diffie-Hellman key agreement is based on mathematical properties of prime order groups. Firstly, $\mathbb{Z}_p^*$ is a group of size p of integers $\{1, \dots, p\}$. If the size of this group is prime, then for any element in this set g , we say g

generates $\mathbb{Z}_p^*$. This simply means that if we multiply g by itself, we can land on any element in this set. We say for any $a \in \mathbb{Z}_p^*$:

$$a = g^k \pmod{p}$$

for some integer k .

The discrete log problem says that for arbitrary $a \in \mathbb{Z}_p^*$, it is hard to find k such that $g^k \pmod{p} = a$.

2.2.2 Protocol

The protocol is as follows.

1. Alice announces large prime p and generator g of $\mathbb{Z}_p^*$
2. Alice and Bob select random elements from the group a and b respectively and send $A = g^a, B = g^b$ to the other party.
3. The generated key they share is g^{ab} which can be calculated by taking their selected element and calculating A^b, B^a .

2.3 Semantic Security

Semantic Security define properties on protocols.

- Correctness: Decryption on a valid ciphertext always produces the message that was encrypted
- Security: Given two messages, the probability that any PPT adversary can tell the difference between the produced ciphertexts of the two is negligible.

2.4 El Gamal Public Key Encryption

The El Gamal Encryption scheme is performed over some large finite group of prime order with chosen generator g as follows:

1. $\text{Keygen}(1^n, R) \rightarrow pK = (G, p, g, g^x), sK = x$ where x is a random group element, G is the group, p is the prime order of the group, and g is a generator.
2. $\text{Enc}(m, pK)$ chooses a random group element r and sends $(g^r, pK^r \cdot m)$

3. $\text{Dec}(c', sK, pK)$ takes $k = (g^r)^{sK}$ and takes the second part of the ciphertext to generate $m = \frac{pK^r \cdot m}{(g^r)^{sK}} = m$

This scheme is secure under the assumption that decisional Diffie Hellman (DDH) is hard. This problem says that not PPT adversary can tell the following two outputs apart with non-negligible probability:

1. $\langle G, p, g, g^x, g^y, g^{xy} \rangle$
2. $\langle G, p, g, g^x, g^y, g^r \rangle$

Where G is a group, p is the size of the group which must be a large prime, g is a generator of the group and x, y, r are random group elements.

Proof of Security: We consider an adversary A that can break the semantic security of this protocol. This means that it must be able to distinguish two ciphertexts with non-negligible probability. The adversary A' that can break DDH given oracle access to A works as follows:

1. A' receives $\langle G, p, g, g^x, g^y, g^c \rangle$ where c is either xy or a random field element r .
2. A' sends A (G, p, g, g^x) which represents its public key.
3. A sends A' two messages m_0, m_1 and A' flips a bit b and returns ciphertext $(g^y, g^c \cdot m_b)$.
4. A' outputs 1 if A outputs b .

Here, if it is the case that $c = xy$ then A 's view is exactly the El gamal protocol, so A' guesses that $g^c = g^{xy}$ correctly with probability ϵ . In the other case, $g^c \cdot m_b$ is a completely random element in the group, so the probability that it gets the bit correct in this case is exactly $1/2$. Since the probability that the algorithm outputs 1 in each case differ by a non-negligible amount this implies that A' can differentiate between the two worlds with non-negligible advantage giving us a contradiction.